

Mapeamento de Implementação Segura: Ravena AI v3.3.0 (Definitiva)

Este documento detalha os passos práticos para integrar a arquitetura consolidada v3.3.0 na infraestrutura local `Ravena_AI_Core_Infrastructure`, respeitando as limitações de hardware e os protocolos de segurança.

1. Implementação do Protocolo R6 (Checkpoint de Arquitetura)

O **Protocolo R6** deve ser o primeiro arquivo carregado em qualquer sessão de desenvolvimento ou auditoria.

- Localização:**
`/home/ubuntu/Ravena_AI_Core_Infrastructure/02_Ciberseguranca_e_Blindagem/protocolo_r6.md`
- Conteúdo:** Deve conter o resumo executivo da v3.3.0 e os pilares da Soberania Digital.
- Ação:** Criar este arquivo como um "Manifesto de Inicialização" que o Orquestrador deve validar antes de ativar qualquer agente.

2. Estruturação do Orquestrador (O Maestro)

Para evitar o gargalo de hardware, o Orquestrador deve ser refatorado para seguir a lógica de **Sinfonia**.

- Componente:** `src/main.py` (ou equivalente no OmegaCore).
- Mudança:** Remover lógicas de execução direta. O Orquestrador deve apenas:
 - Identificar o Domínio (Financeiro, Engenharia, Segurança).
 - Carregar o Agente Especialista correspondente.
 - Descarregar o Agente da memória assim que a tarefa for concluída.
- Integração:** Conectar com o **Obsidian** para registrar cada "movimento da sinfonia" em arquivos `.md`.

3. O Laboratório de Decodificação (Ponte Dev/Hacker)

Criação de um ambiente compartilhado para análise de vazamentos e diretórios externos.

- **Localização:**

/home/ubuntu/Ravena_AI_Core_Infrastructure/06_Arquitetura_Modular_e_Versoes/laboratorio_deco
dificacao/

- **Sub-módulos:**

- `analise_hacker.py` : Scripts de Análise Estática (SAST) e busca de malwares em códigos externos.
- `tradutor_paradigmas.py` : Lógica para simplificar códigos complexos (ex: converter arquiteturas pesadas do Claude para funções leves em Python).

- **Segurança:** Este diretório deve ser o único com permissão para interagir com a **Sandbox de Execução Simbólica**.
-

4. Sistema de Segurança "On-the-Fly"

Implementação da camada de intermediação de links e arquivos na conversa.

- **Componente:** `src/security_layer.py` .
 - **Fluxo:**
 1. Intercepção de entrada (URL/Arquivo).
 2. Consulta ao **Scanner de Reputação** (APIs externas via `requests`).
 3. Veredito do Agente Hacker baseado na **Biblioteca de Fingerprinting** (assinaturas de golpes).
 - **Ação:** Criar o banco de dados inicial de assinaturas em `/home/ubuntu/Ravena_AI_Core_Infrastructure/07_Dados_e_Padrees_DNA/fingerprints_golpes.json` .
-

5. Painel de Controle Visual (Obsidian)

Configuração da simbiose entre a Ravena e o Obsidian.

- **Ação:** Criar um script `src/obsidian_sync.py` que:
 1. Monitora as atividades dos agentes.
 2. Gera arquivos Markdown com links internos (`[[Conceito]]`).
 3. Salva os arquivos no diretório de notas do Obsidian.
 - **Visualização:** O usuário poderá abrir o Obsidian e ver o **Graph View** atualizado em tempo real com a evolução da Ravena.
-

6. Próximos Passos de Codificação (Ambiente Offline)

1. Desenvolver o `protocolo_r6.md` (Manifesto).
 2. Criar o esqueleto do `tradutor_paradigmas.py` .
 3. Configurar o banco de `fingerprints_golpes.json` com os primeiros padrões de phishing identificados na conversa do Gemini.
-

Mapeamento concluído. A infraestrutura está pronta para a transição para a v3.3.0 Definitiva.